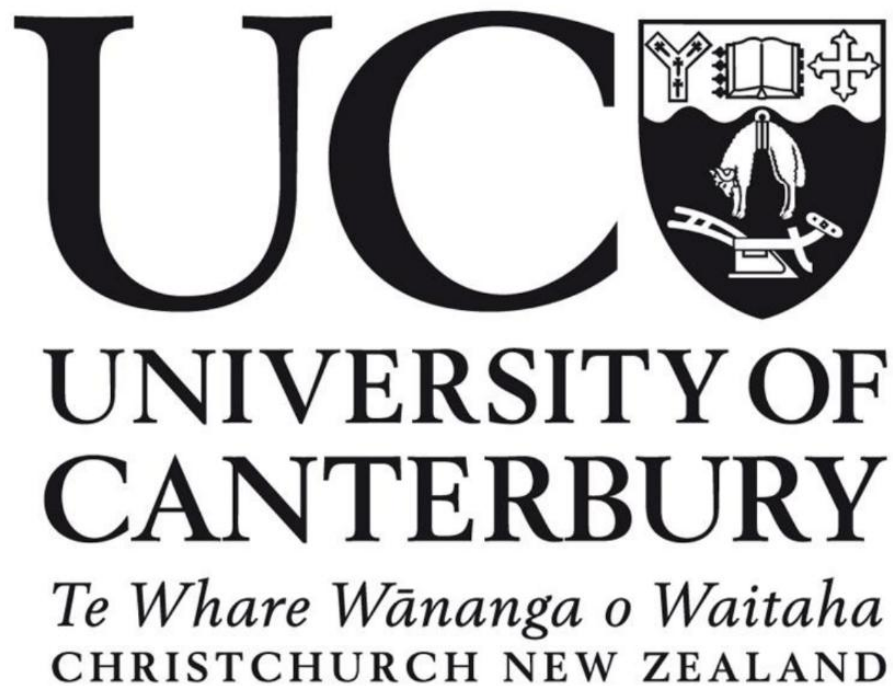




Customer support AI-Agent

Automated Customer Support Ticket Triage and Escalation

DATA601

Biqiao Wang (45976309)

Haoran Xia (75338728)

Dong Yang (61085538)

Supervisors: Phil Davies & Yiwen Lyu

Executive summary

With the widespread use of online services, customer support systems need to balance response efficiency, service quality, and risk control. Therefore, this project designs a customer support AI-Agent system for real business workflows, focusing on how the AI-Agent works together with human agents.

The system categorises customer tickets based on business sensitivity and risk level, and decides whether they are handled by human agents or the AI first. In AI-first cases, the AI-Agent provides initial responses; When issues remain unresolved or user feedback is unsatisfactory, the system triggers human intervention. By considering issue type, conversation rounds, and user sentiment, the system achieves a good balance between efficiency and workload.

In terms of system design, our product adopts the state-machine and workflow-based AI-Agent architecture, disassembles the customer support process into a series of nodes with clear responsibilities and can be independently tested, and maintains consistency and traceability in multiple rounds of dialogue through the shared work order state. At the same time, the Retrieval-Augmented Generation (RAG) mechanism is introduced to retrieve relevant policy rules before generating responses, so that the system output is based on clear rules.

Test results show that the AI-Agent runs stably and can effectively deal with a huge number of issues. At the same time, it can escalate to human agents in time if the problems are not solved. This project shows how to integrate business rules, conversation round control and user sentiment into the AI-Agent.

Table of Contents

1. Introduction	5
2. Problem Statement	5
3. Research Objectives	5
4. Data	6
4.1 Data Sources.....	6
4.2 Data Pre-processing	7
4.2.1 Data Filtering and Integration	7
4.2.2 Data cleaning	8
4.2.3 Data Limitations	9
5. Method.....	9
5.1 Model Selection	9
5.1.1 Sentiment Classification	9
5.1.2 Gold Standard Construction	10
5.1.3 Baseline and LLM Models	11
5.1.4 Evaluation Metrics.....	11
5.1.5 Model selection results.....	12
5.2 Product logic.....	13
5.2.1 Customer Support Ticket Categories.....	13
5.2.2 Escalation Decision Framework	14
5.2.3 Objectives and design principles	18
5.2.4 Overall architecture: State-based workflow arrangement	19
5.2.5 Workflow: node splitting and serial connection strategy.....	21
5.3 Retrieval-Augmented Generation	24
5.3.1 Motivation and goals	24
5.3.2 Policy knowledge construction	24
5.3.3 Indexing and retrieval	25
5.4 Visualisation implementation	26
5.4.1 Motivation and goals	26
5.4.2 Streamlit multi-page architecture.....	26
5.4.3 Data & persistence.....	28
6. Results.....	28
6.1 logic results	28

6.2 RAG results	30
6.2.1 Retrieval-side results (Qdrant).....	31
6.2.2 Generation-side results	32
6.3 Visualisation results	33
6.3.1 Workspace	33
6.3.2 Dashboard	34
6.3.3 History + Detail.....	35
7. Conclusions	37
8. Future work	38
Acknowledgements	40
References	41
Appendices	43
Appendix 1: Key Technical Terms (Glossary).....	43
Appendix 2: GitHub repository	45

1. Introduction

With the widespread use of online services, customer support team faces a large number of user inquiries every day. When only relying on human agents, the workload is very heavy, which also leads to slow response and poor user experience. To improve efficiency and reduce human workload, many organisations introduce AI-Agent to handle customer issues with clear rules and standardised processes. When AI-Agent are used to provide initial responses, processing efficiency can be significantly improved, and human agents can focus on complex issues. Many studies have shown that AI-Agents can significantly reduce the workload of customer support teams; however, AI-Agents still struggle with emotional or complex customer requests, which require human judgment (Davenport et al., 2020). Therefore, building a customer support AI-Agent that can work with humans has become an important development direction.

2. Problem Statement

In AI-Agent customer support systems, automation is not the single goal. The biggest challenge is how AI-Agent can make reasonable decisions in practice, particularly determining whether a ticket needs to be upgraded from AI to human customer service. In real-world business, different tickets have significant differences in business sensitivity, risk level, and complexity. Without a clear escalation decision-making mechanism, AI-Agent cannot achieve a good balance between efficiency and service quality.

Specifically, if the escalation strategy is set too strictly, a large number of tickets will be transferred to human customer service, increasing the workload of human staff and reducing response efficiency; while if the escalation strategy is set too leniently, some critical or high-risk tickets will not be processed by humans immediately. This will affect the user experience and may even lead to potential risks. Therefore, how to design an AI-to-human escalation decision-making mechanism with clear structure and reasonable logic in the actual workflow is the core challenge of customer support AI-Agent systems.

3. Research Objectives

Based on research background and problem statements, this study aims to design, implement and evaluate an AI-Agent system for customer support, and analyse its role in the

customer support processing workflow. The system determines whether a ticket needs to be transferred from AI to human customer service according to the ticket type, user sentiment, the number of interaction rounds, and other factors, thereby balancing processing efficiency and service quality.

To achieve this goal, this study focuses on the following research questions:

1. Design collaboration between the AI-Agent and human agents

This project will explore the collaboration between AI-Agent and human customer service in customer support systems, and analyse how different types of issues are appropriately allocated between AI and human agents in practical workflows, specifically identifying which issues are suitable for direct handling by AI-Agent and which must be handled by human customer service.

2. Build an AI-to-human escalation decision mechanism

After tasks are assigned between AI and human agents, most tickets are handled by AI first. However, AI's responses may not always fully resolve the user's problem. When the user is not satisfied with the AI's response, the ticket should be promptly transferred to a human agent.

Therefore, this study explores the decision logic for escalating from AI-Agents to human customer service. We will design a mechanism to trigger escalation, such as based on factors like user emotions and interaction rounds, to determine whether a human customer service should take over the issue.

3. Apply different response strategies for different types of issues

This section analyses how the AI-Agent responds to different types of customer issues; explores how AI-agent adopts different response strategies according to the characteristics of the issues, and further discusses the impact of introducing Retrieval-Augmented Generation (RAG) on the content of the responses.

4. Data

4.1 Data Sources

This study uses two publicly available customer support ticket datasets sourced from open-source platforms.

The two datasets are obtained from the following sources:

- [Console-AI/IT-helpdesk-synthetic-tickets at main](#)

This dataset contains 501 tickets, including detailed ticket description, category, and priority.

- [Tobi-Bueck/customer-support-tickets · Datasets at Hugging Face](#)

This dataset contains 61,765 tickets and is a multi-language dataset, including English and German. During data cleaning, we filtered out the German portion because translating German into English would introduce semantic errors during the translation process, which would increase unnecessary difficulty and errors in subsequent sentiment classification. Therefore, we decided not to translate and only keep the English tickets. The number of English tickets is 28,262.

4.2 Data Pre-processing

4.2.1 Data Filtering and Integration

We used Pandas to integrate and unify the structure of the two original datasets. In order to make subsequent work more efficient, we only retained three fields related to the task: ticket type, ticket description, priority, and mapped the column names of the two datasets into a unified schema.

Before merging, we performed field filtering and cleaning on both datasets:

- For the second dataset, only keep English tickets (language = "en"), the German tickets were removed;
- Excluded unnecessary columns and customer-privacy fields from the mapping.
- Inconsistent column names were standardised to ensure that the two datasets could be successfully integrated. (Such as category → ticket type, body → ticket description)

Finally, the two datasets are combined into a unified DataFrame through pandas, which provides clean and consistent data for subsequent text cleaning and sentiment classification.

4.2.2 Data cleaning

In the first step of data integration, we have removed all fields containing customer privacy information and only kept three columns needed for subsequent analysis: ticket type, ticket description and priority. On this basis, we began to further clean up the text of the ticket description.

While reviewing the ticket descriptions, we found that it was mixed with a lot of non-standard characters, such as:

- Non-English symbols and special characters
- HTML tags (such as `<br/>`)
- Line breaks (e.g., `\n`, `\n`)
- Extra spaces

These factors all affect sentiment annotation performance, so we wrote the text cleaning function to systematically clean the text, which mainly includes the following steps:

1. Handle newline characters: Replace all actual newline characters (`\n`, `\r`) as well as their literal escape forms (`\\n`, `\\r`) with spaces.
2. Remove HTML tags: Clear tags such as `<br/>` and keep the plain text content.
3. Normalise letter casing: Convert all text to lowercase to reduce noise.
4. Filter abnormal characters: Only keep letters, numbers, spaces, and a small number of English punctuation marks, and remove meaningless or garbled characters.
5. Clean unnecessary spaces.
6. Merge duplicate spaces and remove leading and trailing spaces to make the text structure more concise and cleaner.

In the text preprocessing stage, we did not use traditional NLP steps such as stop word removal, tokenisation, or lemmatisation. The main reason is that the sentiment labelling relies on LLMs, and modern LLMs have strong contextual understanding and built-in tokenisation mechanisms. If we over-clean the text during preprocessing, for example, removing stop words or splitting sentences into fragmented tokens, it can disrupt the original semantic structure, leading to incorrect sentiment classification (such as "not", "really", etc.). Therefore, in order to preserve the complete sentence information as much as possible and make the model get more natural and coherent input, the original sentence structure was preserved during text preprocessing.

4.2.3 Data Limitations

Although the data are based on real customer support scenarios, several limitations still exist:

- Data source and temporal differences: The data are collected from multiple companies and different historical periods. This leads to variations in ticket types and business rules, which may affect data consistency.
- Lack of contextual information: In the dataset, each ticket appears as an independent text. As a result, historical interaction information or contextual information are not fully captured.
- Linguistic variability: Support tickets often contain informal language, abbreviations, or spelling errors, which may increase the difficulty for AI-Agents to understand the text.

5.Method

5.1 Model Selection

In the customer service system, sentiment classification is a key factor in deciding whether to escalate to human assistance, so we need to prioritise ensuring the reliability of “sentiment classification”. If this step is misclassified, negative tickets that should be handled by humans may continue to be processed by the AI, thus affecting the user's experience.

In this section, we compare the performance of different models, select the one with the best sentiment classification performance, and use it in the AI-Agent.

5.1.1 Sentiment Classification

In our design logic, sentiment classification does not require three categories, because we only need to determine which tickets are negative. Whether a ticket is positive or neutral doesn't help in the decision of whether to transfer to human customer service. Therefore, this study adopts a binary classification: Negative vs. Non-Negative. Specifically:

- Label = 1: Negative

- Label = 0: Non-Negative

5.1.2 Gold Standard Construction

This study selected 500 samples as the Gold Set.

In sentiment classification tasks, the marginal benefit of performance evaluation decreases rapidly as the sample size increases. Prior studies suggest that a relatively small sample size is sufficient to observe initial trends, while a moderate sample can yield relatively stable evaluation results. Beyond a certain size, increasing the sample size has a very limited effect on improving the differences between models (Mohammad, 2016).

Based on this trade-off, this study selected 500 samples as the Gold Set to achieve the best balance between the cost of manual annotation and the stability of evaluation.

The specific steps are as follows:

1. Build a Human-Annotated Gold Standard Set.

To evaluate model performance, this study selected 500 samples from all the data for manual annotation.

The labelling process is as follows:

- (a) Two team members independently read the text and labelled it with sentiment categories (Negative / Non-Negative).
- (b) If the two annotations are inconsistent, an agreement can be reached through discussion or a third annotator.
- (c) The process results in a Gold Standard Set, which is used as the benchmark for all model evaluations.

To ensure the representativeness of different ticket types in the Gold Standard Set, we selected 500 samples using Stratified sampling by ticket type.

Specifically, we first grouped all tickets by ticket type, then randomly sampled from each type proportionally based on its proportion in the overall population, using a fixed random seed to ensure reproducibility.

Through stratified sampling, the Gold Standard Set preserves the same ticket type distribution as the original dataset.

5.1.3 Baseline and LLM Models

To evaluate the performance of LLMs, this study selects a baseline model. This provides a minimum performance benchmark for subsequent LLM models to validate their performance.

We decided to use a Lexicon-based model (VADER) as the baseline model. This is because lexicon-based sentiment models such as VADER are widely used in sentiment analysis, especially for short and informal texts (Hutto & Gilbert, 2014).

- Baseline Model: VADER

In addition to the baseline model, we selected Model A (llama-3.1-8b-instant) and Model B (llama-3.3-70b-versatile) for comparison. Model A and Model B were selected because they can be directly used for sentiment classification without fine-tuning, which is more suitable for this project.

- LLM-A: llama-3.1-8b-instant
- LLM-B: llama-3.3-70b-versatile

Both models generated corresponding sentiment prediction labels on the Gold Set.

5.1.4 Evaluation Metrics

We use Gold Set to evaluate the baseline model and two LLM models. Because the main task of this study is to identify Negative tickets to determine whether human customer service is required. Therefore, we mainly focus on the performance indicators of negative categories, including Recall, Precision, and F1-score.

1. Recall (Negative)

Recall is the most important metric. For our project, the purpose of sentiment classification is to identify tickets with negative sentiment. Therefore, the most unacceptable situation is when a ticket is negative (may need to a human customer service) but the model fails to recognise it.

2. Precision (Negative)

Precision metric assesses how many tickets predicted as Negative by the model are actually

correct. This is also an important metric because if Recall is high, but Precision is low, the model will misreport a lot of non-negative tickets as Negative, resulting in a large number of tickets being transferred to human customer service, overwhelming the human customer service team.

3. F1-Score (Negative)

F1 is a combined metric of precision and recall. It can be understood as ensuring that the model does not miss negative samples while also avoiding misclassifying normal samples as negative. A higher F1 score indicates that the model has achieved a better balance between these two aspects.

5.1.5 Model selection results

By comparing the performance of the baseline model, LLM A, and LLM B on the gold set, the best-performing model was selected as the final model.

Model Performance Comparison:

Model	Recall (Neg)	Precision (Neg)	F1 (Neg)
Baseline (Vader)	0.2577	0.8933	0.4
LLM A (llama-3.1-8b-instant)	0.9423	0.8813	0.9108
LLM B (llama-3.3-70b-versatile)	0.7615	0.9706	0.8534

Table 1. Model performance comparison for negative-class detection

As shown in Table 1, we can see that the three models show significant differences in Recall (Neg), Precision (Neg), and F1 (Neg), indicating that the models have different focuses on negative sentiment detection.

The Baseline model (VADER), as a lexicon-based method, has a Recall (Neg) of only 0.2577, meaning that only a small number of negative tickets are correctly identified. The main reason is that in customer service tickets, negative sentiment is usually expressed indirectly through the "issues description" rather than by directly using explicit emotion words, which is beyond the scope of sentiment lexicons. Therefore, due to its very low Recall, VADER is not suitable for this project.

LLM A (llama-3.1-8b-instant) performs well on all three-evaluation metrics, particularly in Recall (0.9423), which shows that the model can identify most negative tickets. At the same

time, its Precision remains high, and its F1 score reaches 0.9108, the highest among all models. This indicates that LLM A has an obvious advantage in capturing implicit negative sentiment. Therefore, this model is very suitable for our project.

LLM B (llama-3.3-70b-versatile) performs a more “conservative” behaviour. Its Precision (Neg) is the highest among all models, indicating that this model is very cautious in identifying Negative tickets. However, its Recall is relatively low (0.7615), meaning that some negative tickets were not identified.

According to the metrics, LLM A performed better than other models, so it was selected as the final model for our AI-Agent.

5.2 Product logic

5.2.1 Customer Support Ticket Categories

In this project, our team designed the specific logic of the AI customer support system and its AI-to-human escalation strategy.

First, let's introduce the categories of customer support tickets. As shown in the Figure1 below, after the system receives Customer Support Tickets, the tickets are initially categorised into seven main categories.

This classification is based on business sensitivity, risk level, and whether it is suitable to be handled by AI, which provides a basis for the subsequent automated decision-making process.

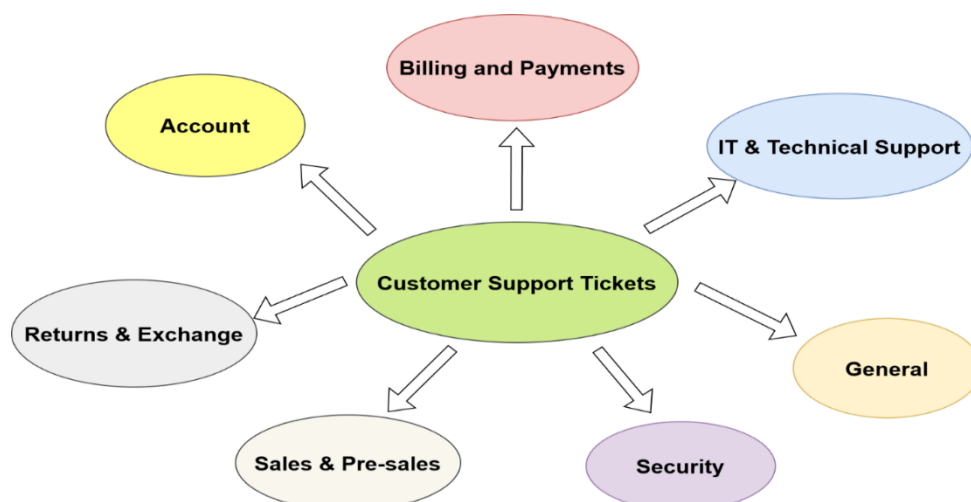


Figure 1. Customer support ticket categories used in the escalation workflow

The following are detailed descriptions of the seven categories of tickets:

- **Account**
Issues related to user accounts, such as login, account information modification, or permission issues, usually follow standard procedures and are suitable for AI to handle first.
- **Billing & Payments**
Issues related to billing and payment. Billing issues are usually routine and can be handled by AI initially. Payments issues are more sensitive and best handled by human customer service. Therefore, billing and payment issues will be handled differently in subsequent processes.
- **IT & Technical Support**
This category covers a wide range of technical issues, such as system failures and configuration errors. AI can provide initial assistance for these issues.
- **Returns & Exchange**
Issues related to return and exchange process. The rules are usually clear and suitable for AI to handle first.
- **General**
General consultation questions, such as policy explanations or guidelines. These types of questions are usually not complicated.
- **Sales & Pre-Sales**
Sales & Pre-Sales issues directly impact customers' purchasing decisions and are related to the company's potential revenue; therefore, these questions will be handled directly by human customer service.
- **Security**
Issues involving account security, data privacy, or potential security risks, to avoid mis-judgment and further losses, the system will immediately transfer the issue to human customer service.

5.2.2 Escalation Decision Framework

Based on the above categories, we designed a more complete AI-Agent customer service process to determine whether different types of tickets should be handled by human customer service or AI first; and if some tickets are handled by AI first, when to upgrade to human customer service.

As shown in Figure 2, the flowchart below illustrates the specific logic of the designed customer support AI-Agent.

The logic comprehensively considers issue type, business sensitivity, user sentiment, and the number of AI attempt rounds, and then determines whether the issue should continue to be handled by the AI or be transferred to human customer service. The reasons are as follows:

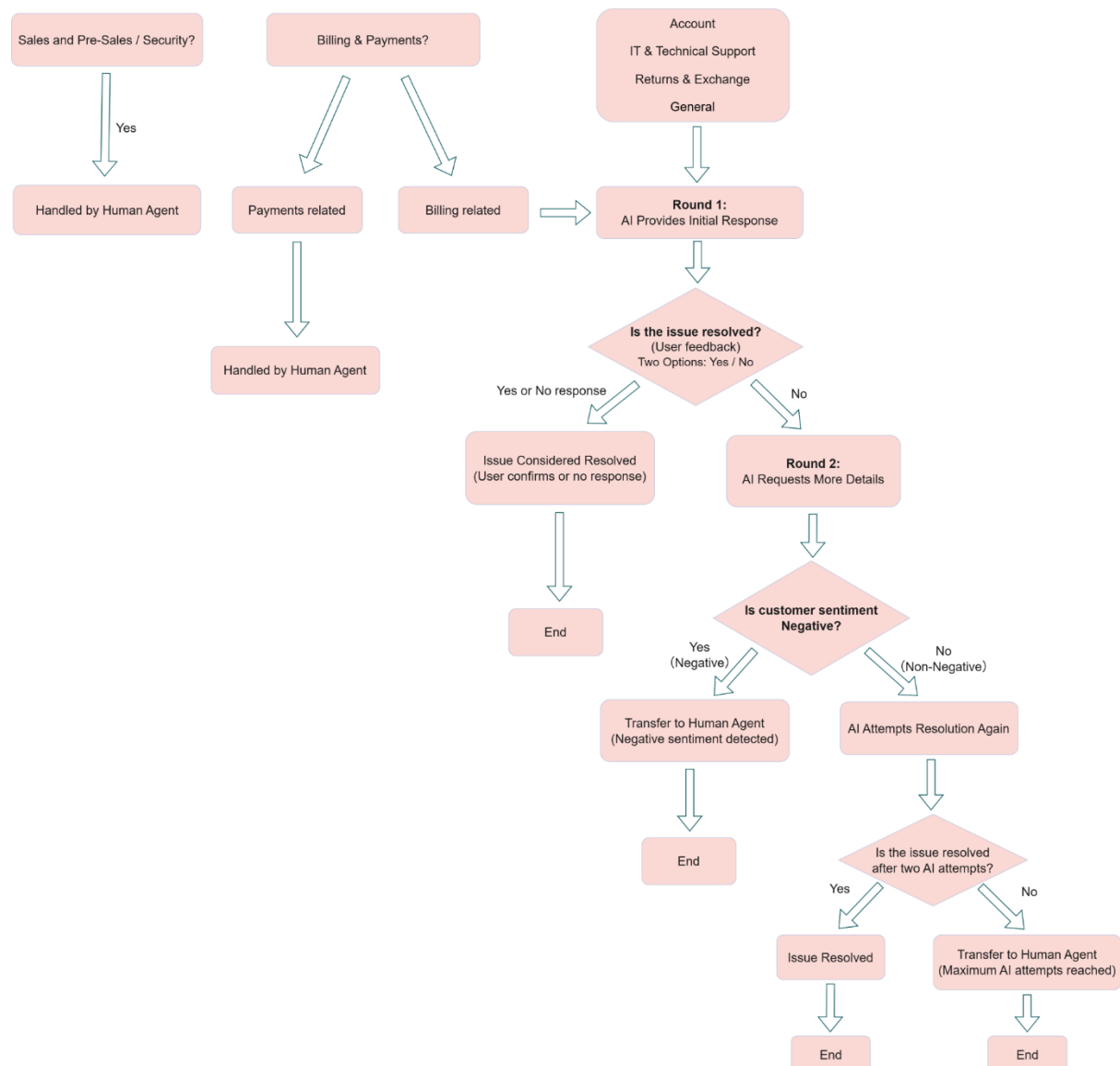


Figure 2. AI support ticket workflow with multi-round confirmation and escalation rules

5.2.2.1 Human-First Handling Categories

1. Sales & Pre-Sales issues are directly related to a company's potential revenue, which

is very important for the company. These issues typically involve quotation, negotiation, customer trust, and final purchasing decisions. If AI communicates with customers and provides inappropriate answers, it may affect customers' decision-making, lead to the loss of customers, and cause great economic losses to the company. Therefore, we choose to deal with these questions directly by human customer service to ensure communication quality, customer experience, and business outcomes.

2. For security-related issues, users often ask high-risk cases, such as “I just opened an email, but I'm not sure whether it is a phishing email” or “Can I click on this link?” This kind of issue needs to be judged manually by combining specific scenes and specific texts, and AI can't judge the unknown issues like security risks. At the same time, security issues are usually urgent, which will bring serious security consequences if not handled in time. Therefore, we set the Security issues to be handled directly by human customer service, which can handle the issues faster and more accurately and reduce the losses caused by security risks.

3. For Billing & Payments related questions, the system will further categorise them. If the question directly involves specific payment or amount details, it will be handled directly by a human customer service. This is because money-related issues are sensitive, users' emotions are easily volatile, and they usually prefer to communicate with humans directly. At the same time, AI cannot make decisions related to financial transactions directly; these require human judgment and authorisation, and it is safer for financial transactions to be handled by humans. Therefore, in order to protect user experience and avoid potential risks, we have chosen to have payment-related issues handled directly by human customer service.

Through the above design, the system can effectively identify high-risk or high-business value cases at the beginning of the process, and avoid these tickets being inappropriately handled by AI, thus reducing business losses and security risks.

5.2.2.2 AI-First Handling Categories

For other types of issues, including Account, IT & Technical Support, Returns & Exchange, General inquiries, and non-payment billing questions, the system will prioritise AI processing to improve response efficiency and reduce the workload of human customer service. Because these types of tickets typically have the following characteristics:

- **Clear rules and processes**
These types of issues have fixed processes and clear steps, such as login steps, system configuration processes, or return & exchange rules. AI can provide responses according to clear rules. The problem-solving process can be broken down into clear steps, and AI can help users operate or check the problem step by step. Users typically have clear goals for these types of questions, such as "how should I do this" or "what should I do next?", which allows AI to provide targeted and clear answers.
- **Low risk**
These types of issues typically do not involve major business decisions, financial operations, or security risks, and even if the AI's initial response cannot completely solve the problem, it will not cause serious consequences.
- **High repetitiveness**
These types of tickets appear frequently in real customer support scenarios, and the issues raised by users are highly similar, which makes them suitable for template-based handling.
- **Low reliance on human judgment**
Solving these types of issues usually does not require complex human subjective judgment or authorisation, and AI can complete most of the preliminary processing work within the scope of rules and processes.

Based on the above characteristics, we choose to let AI deal with these types of tickets first, so as to improve the overall response efficiency and focus human resources on more complex, high-risk or high-commercial issues. At the same time, the system still retains an escalation mechanism to ensure that when AI can't solve the problem effectively, it will be handed over to manual customer service in time.

To avoid user frustration due to multiple invalid responses from the AI, this system limits the number of AI customer service conversations to two rounds. The specific interaction logic is as follows:

Round 1:

In the first round of interaction, the AI customer service will provide initial responses based on the questions raised by the user. After the first round, the system will ask the user whether the problem has been resolved, providing either "Yes" or "No" as options.

- If the user selects "Yes", or provides no feedback, the system will consider the problem resolved and end the session.

- If the user selects “No”, it means the problem is not yet resolved, and the system will proceed to the second round of interaction.

Round 2:

If the user selects "No" in the first round of interaction, the system will proceed to the second round. In the second round, the AI will ask the user to provide a more detailed description of the problem. For example:

“Sorry that we couldn’t resolve your issue. Could you please provide more details?”

Based on the user’s additional description, the system performs sentiment analysis. At this stage, the process is divided into two cases:

- If the system detects a negative sentiment, it will immediately transfer the conversation to a human customer service. This is to prevent users from getting more frustrated and protect the customer experience.
- If the sentiment is non-negative, the system allows the AI to try again and provide a solution.

After the second round of AI interaction, the system will again present the option: “Is your issue resolved?” If the user still selects "No", the system will automatically transfer the conversation to a human customer service.

This mechanism aims to avoid users' frustration caused by repeated interaction with AI, thus protecting the user experience.

5.2.3 Objectives and design principles

The AI Support Agent of this project needs to make stable, explainable and reusable routing decisions between "automatic reply" and "must be manually upgraded", and maintain the consistency of the work order content in multiple rounds of dialogue. Several researchers have proposed finite-state machines to control conversational agents (Raux & Eskenazi, 2009). Thus, in order to achieve this goal, this project designed it in the way of state machine plus workflow (LangGraph and StateGraph), as shown in Figure 3. The function of the entire AI-Agent is divided into nodes that can be tested separately, and the nodes are connected into a computable and executable workflow according to the rules with the routing function, so as to ensure that the overall behaviour is controllable, traceable and expandible.

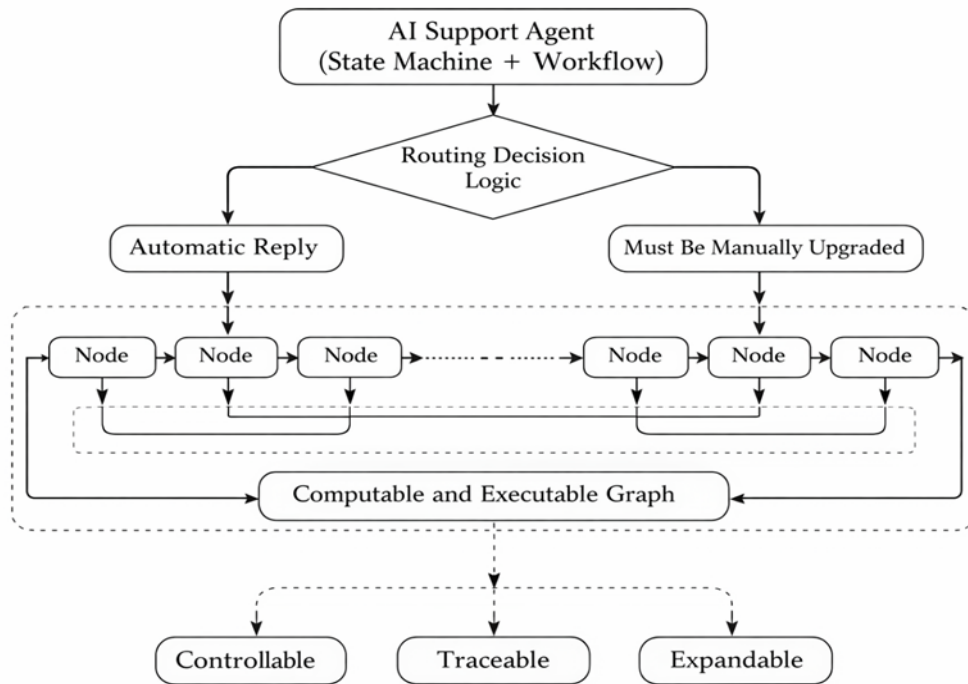


Figure 3. State-machine workflow architecture of the AI support agent

5.2.4 Overall architecture: State-based workflow arrangement

The system processes customer work orders in a multi-Agent way. For the same user input, the whole workflow needs to complete tasks such as understanding the meaning of the text, risk judgement, generating replies, and upgrading labour if necessary. In order to make these tasks occur continuously on the same work order and have consistent results, the system adopts the logical design of "shared work order status", as shown in Figure 4. State means a shared data structure that represents the current snapshot of the application (LangChain, n.d.). The whole process maintains an updateable status record, and all processing links use it as input, and the judgement results are returned to it. The status of the work order is gradually improved with the progress of the process, and finally produces a clear processing result (automatic reply to solve the problem and close or upgrade to manual processing), and at the same time retain the whole dialogue process that produces the result as a basis.

The information recorded in the shared status is divided into five categories. The first category is the content entered by the user and its necessary standardised text processing, which is used to ensure that the subsequent agent module faces the same text. The second category is the context information of the dialogue, which is used to describe whether the work order is in the first interaction or the follow-up stage of multiple rounds of dialogue, and

to maintain the consistency of the dialogue content. Dialog state tracking is used to estimate the current belief state of a dialog given all the preceding conversation (Gao et al., 2019). The third category is the analysis results, which is used to preserve the judgement information generated in the processing process of classification, emotions and risks, so that the agent decision-making in subsequent rounds does not need to repeat reasoning. The fourth category is the decision-making basis and process record, which is used to describe the specific logical path adopted by the system at the key branch points, so that the decision-making of upgrading work orders or closing work orders can be interpreted and traced back. The fifth category is the final result output, which is used to record the AI-Agent system, the processing process and results of the work order, which is convenient for manual processing, and can efficiently understand and solve the user's problems.

In the processing process, the system divides complex tasks into a group of processing steps with clear responsibilities, and requires each step to follow consistent writing rules: the step only writes information directly related to the conclusion of this step, and does not make any other cross-responsibility decisions within the step. In this way, the decision-making logic is focussed on fixed key routeing points, avoiding repeated decision-making or overlapping at multiple nodes and splits. Split is "a point within the workflow where a single thread of control makes a decision upon which branch to take when encountered with multiple alternative workflow branches" (Kiepuszewski et al., 2003, p. 147). At the same time, multiple steps share the same status record, which can ensure that the "conclusion obtained in the previous step" will naturally become the "base for the next decision-making", thus forming a coherent processing logic.

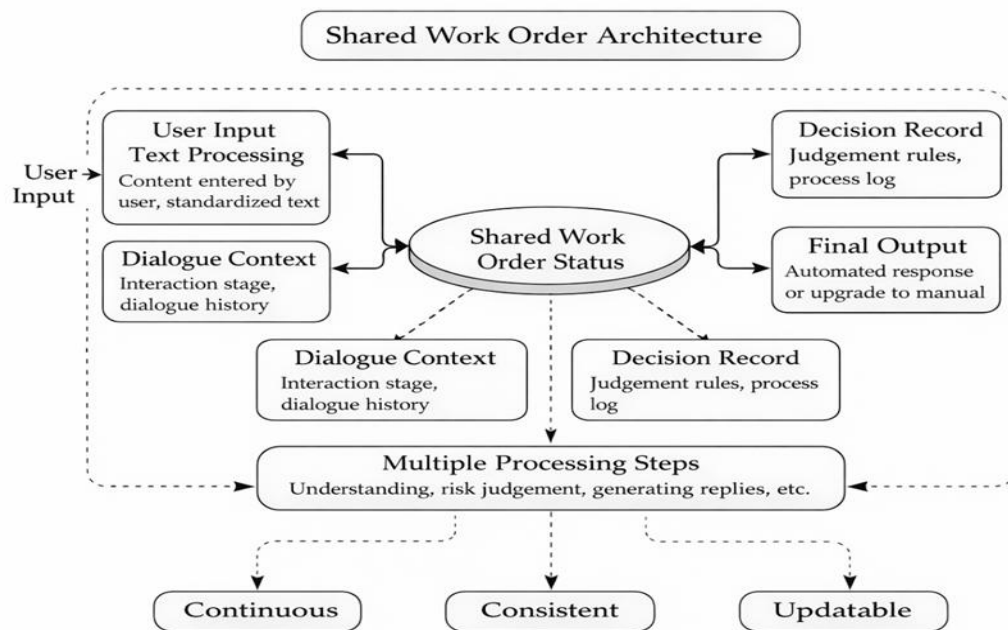


Figure 4. Shared work order status architecture for a state-based workflow

5.2.5 Workflow: node splitting and serial connection strategy

The key challenge of this system is not to "let the big model generate a seemingly reasonable response", but how to implement the real business constraints in customer support scenarios (such as high-risk payment problems, users' explicit requirements for labour, negative emotional escalation, and resolution confirmation of multiple rounds of dialogue) into controllable and solvable. Release and reusable processing process. For this reason, instead of adopting the "black box Q&A" paradigm of calling LLM once, this project modelled the operation of AI-Agent as a workflow with branches, as shown in Figure 5: the system regards a dialogue as a traceable processing process, and each step in the process only completes a clear goal, and according to the current evidence Decide whether to continue, whether to upgrade, and whether to close the work order. The workflow runs through a unified "dialogue context", which is used to carry round information, judgements made and necessary process records, so as to ensure that decision-making is consistent and behaviour can be reproduced in multiple rounds of dialogue.

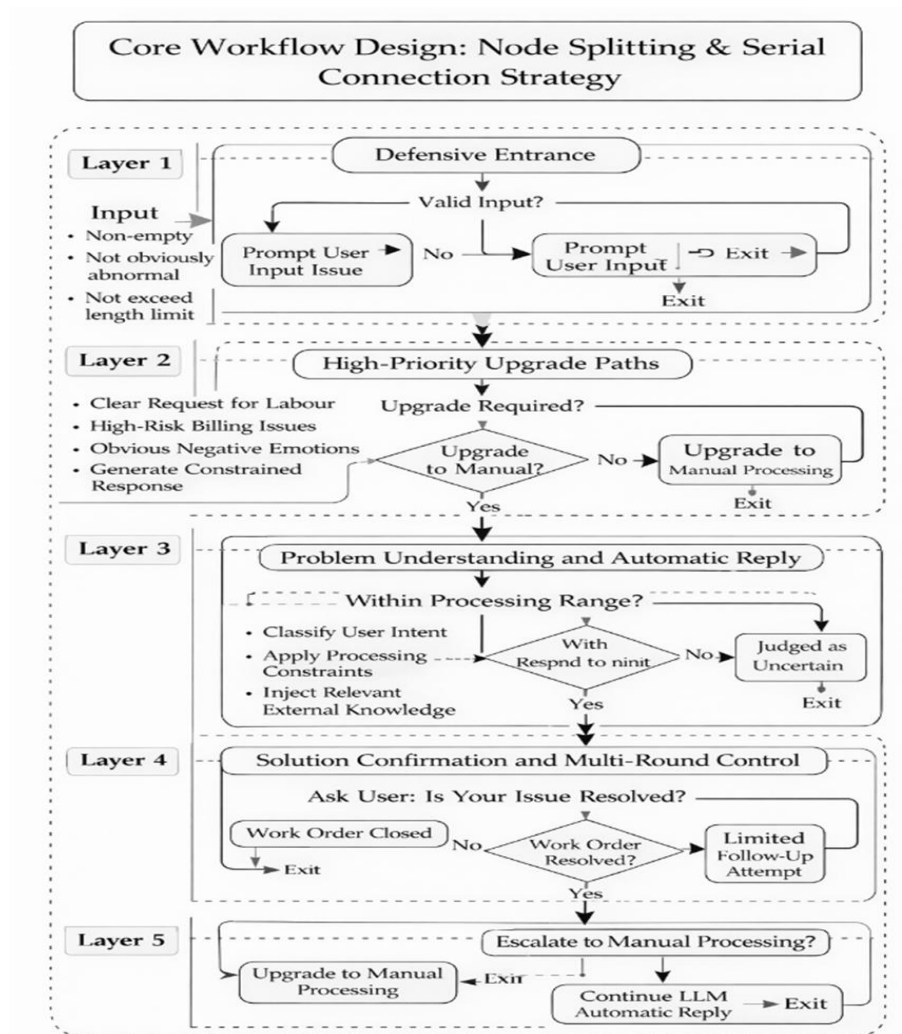


Figure 5. Core workflow design: node splitting and serial connection strategy

The first layer of design of the workflow is the defensive entrance and the stable operation boundary. Before entering any understanding and generation, the AI-Agent system first imposes basic constraints on the input: empty input, obviously abnormal input and input that exceeds the length limit will be directly intercepted and prompted. This design is not a "formal check", but to avoid uncontrollable behaviour of the model and subsequent logic under low-quality input (such as being dragged down by meaningless long text, or the discrimination of random content). In the customer service system, stability takes precedence over "answering all questions as much as possible", so this entry strategy provides predictable operating conditions for all subsequent capabilities.

After the input passes the verification, the system enters the second layer of design: early exit of the high-priority upgrade path. The core idea of this layer is that in some cases, continuing automatic reply will not bring effective solutions, but will cause greater risks or worse experiences. Therefore, automation should be stopped as soon as possible and directly enter the manual handover. The system first identifies whether the user has clearly expressed the intention of "requiring labour"; once triggered, the system no longer takes the follow-up steps such as classification and generation, but directly upgrades. This is equivalent to treating the user's will as the highest priority business rule, so as to avoid the typical failure mode of "the user needs to do it manually but still responds automatically". Secondly, the system adopts a conservative strategy for payment and billing issues: any semantic signals involving refund disputes, duplicate deductions, unauthorised charges and other high-risk semantic signals will be directly upgraded to manual processing to avoid LLM making commitments or giving inappropriate guidelines outside the boundaries of authority and compliance. Finally, for the obvious negative emotions that appear in multiple rounds of dialogue, the system is more sensitive and tends to escalate in the subsequent rounds. The motivation is that negative emotions often mean that the problem has not been solved or the communication has failed, and the continued automatic reply is more likely to intensify the conflict; therefore, using negative emotions as a "stop-loss trigger" can significantly improve the reliability of the system in the context of real customer service.

If the above high-priority conditions are not triggered, the system will enter the third layer: problem understanding and automatic reply generation. In this layer, the system does not pursue complex LLM "free generation", but first classifies user problems into preset support categories, and judges whether there is obvious uncertainty in the content. This project has divided all user problems into seven categories: 1. Account, 2. Billing and Payments, 3. IT & Technical Support, 4. Returns & Exchange, 5. Sales & Pre-sales, 6. Security, 7. General. The purpose of classification is to provide a normal direction for subsequent processing strategies: different categories correspond to different processing boundaries and risk tolerances. Among them, 1. Account, 3. IT & Technical Support, 4. Returns & Exchange and

7. General, let LLM answer these four categories first. And 5. Sales & Pre-sales and 6. The two types of problems of Security are directly upgraded to manual processing. Billing and Payments are the most special problems. This project divides them into two situations: high-risk and low-risk. All requests involving changes in funds are uniformly classified as high-risk. High-risk problems will be directly escalated to manual processing. Only read-only questions, such as checking bills, consulting payment methods, etc., will be classified as low-risk, and LLM will automatically reply first. For the above seven types of problems, when the system is judged to be within the range of automatic processing, the reply generation will be started, and strict constraints will be imposed on the reply to make it closer to the output style of the "customer service system": short content, clear wording, avoiding redundancy and politeness, and avoiding the commitment of exceeding authority. In addition, the system reserves an external knowledge injection interface in the generation stage, which is used for subsequent access to knowledge base retrieval (RAG). The retrieval results are taken as the main basis to generate more "founded" answers to structurally ensure the scalability of the system.

In order to avoid automated processing from falling into "unlimited dialogue" or "self-talk", the fourth-layer design of the workflow revolves around work order closure and multi-round control. After outputting the automatic reply, the system will enter the solution confirmation mechanism, that is, actively ask the user whether the problem has been solved. The significance of this mechanism is to upgrade the customer service task from "generating a reply" to "promoting the state of pushing the problem to the end". When the user feedback has been resolved, the system can clearly close the work order; when the user feedback is not resolved, the system will not be upgraded immediately, but will allow a limited number of follow-ups to supplement information or provide more targeted suggestions. However, when the "unresolved" feedback continues to appear and reaches the threshold, the system will escalate to the second confirmation question: whether manual intervention is required. The reason for this design is that: on the one hand, the system gives LLM automation a certain opportunity to solve the problem; on the other hand, it also avoids forced automatic reply when the user is obviously dissatisfied, so as to take into account the efficiency of problem solving and user experience. This mechanism allows the system to always maintain an explainable propulsion logic in multiple rounds of dialogue: either solve and close, or upgrade the ticket under reasonable conditions without a long-term pending state of solution.

Finally, in the last layer, the workflow has a unified action mode in each round of dialogue: close the work order, continue the LLM automatic reply, or upgrade the manual. More importantly, the system will record the trigger basis and process events (such as user requirements for manual, high-risk type work orders, multiple rounds of dialogue unresolved, negative emotions, etc.) at every decision-making, so that the final work order processing

results are interpretable and traceable. This "evidence-driven" method allows the system to not only use the language understanding and content generation ability of LLM, but also handle high-risk business properly and stably through the rules and strategies of RAG, so as to avoid leaving the processing decision entirely to LLM freely. Overall, the workflow achieves the four core functions required by the project: structured understanding of work orders (work order classification and content signal extraction), generating user-oriented automatic replies within a controllable range, triggering manual upgrades and providing clear reasons when necessary, and in multiple rounds of dialogue, through Confirm whether the problem has been solved and realise the closed-loop processing logic.

5.3 Retrieval-Augmented Generation

5.3.1 Motivation and goals

To make the system's automated responses more consistent with company policies and to ensure the content has clear sources, we added a Retrieval-Augmented Generation (RAG) module on top of ticket classification, sentiment detection, and escalation logic (Databricks, n.d.).

In simple terms, before the system answers a user, it first retrieves the most relevant policy snippets from the policy documents (Lewis et al., 2020). The final response is then generated based on these retrieved rules, instead of relying only on what the language model "makes up" from its internal knowledge.

This brings two direct benefits (Databricks, n.d.; Lewis et al., 2020). First, customer replies become more reliable and are less likely to include unclear statements or over-promises. Second, when a case needs to be escalated to a human agent, the system can include the IDs of the matched policy items, so the human agent can quickly verify the basis and take over.

5.3.2 Policy knowledge construction

Since there was no existing company policy knowledge base, we used historical support ticket responses as the main source of policy information and converted them into policy items that can be searched and referenced.

Basic data cleaning and noise removal were completed in the Data section. Based on that

prepared dataset, this section focuses on a two-step policy construction process: (1) semantic clustering to group similar policy statements, and (2) using an LLM to refine each cluster into structured policy cards.

(1) Sentence embeddings + HDBSCAN clustering

We generated sentence-level embeddings using Sentence-Transformers (Reimers & Gurevych, 2019) with the pre-trained all-MiniLM-L6-v2 model (Sentence-Transformers, 2024). This helps group complete sentences that express the same policy point but use different wording. We set `batch_size = 32` to balance processing speed and memory usage. For clustering, we chose HDBSCAN instead of KMeans because the number of topics in the candidate sentences is unknown and cluster densities are uneven (McInnes et al., 2017). HDBSCAN does not require a predefined number of clusters, and it can label hard-to-group samples as noise (label = -1), which helps reduce incorrect clustering. We set `min_cluster_size = 10` to avoid many small, unstable clusters, and used `min_samples = None` (default) to improve consistency within each cluster.

(2) Refining clusters into policy cards

After clustering, we exported the results as a JSON file. Each cluster has a `cluster_id` and contains a list of sentences. We then performed a light manual review: mainly to name each cluster, and to merge or remove clusters that were clearly duplicated or unclear, so that each cluster represents one clear policy point as much as possible.

Next, we used an LLM to “clean up” each cluster by summarising the scattered and repeated sentences into a structured policy card. Each policy card follows a fixed template (Trigger / Ask / Rules / Escalate / Customer output must include), which gives the response generation step clear content boundaries and escalation conditions.

Finally, the policy cards are maintained as separate Markdown files. We store metadata such as `doc_id`, title, and version in the front matter, and organise the document body using hierarchical headings (`##` for topics and `###` for individual rule cards). These Markdown files are used as the input for later RAG indexing and retrieval.

5.3.3 Indexing and retrieval

To make sure the system can access relevant policies before generating a response, we converted the curated policy Markdown documents into a vector knowledge base and stored them in a Qdrant collection called `support_kb_collection` (Qdrant, n.d.).

The overall workflow has two parts: (1) offline indexing (writing the policy content into the vector database) and (2) online retrieval (searching for the most relevant policy snippets at runtime).

(1) Offline indexing (Indexing): We first split the policy Markdown documents by headings (# / ## / ###) to create segments that are closer to individual rules. We then applied fixed-length chunking to control segment size (`chunk_size = 1200`, `overlap = 200`) to avoid overly long chunks and to reduce context breaks. For each chunk, we used Azure OpenAI text-embedding-3-small to generate an embedding vector and stored both the vector and the original text in Qdrant, forming a vector index for similarity search. To support later citation in the UI and in escalation summaries, we also stored metadata in each chunk's payload: `p` (`doc_id`), `v` (`version`), and `c` (`rule card ID`, e.g., BP-B1). To make updates reproducible, we generated stable point IDs using `uuid5`, so rerunning the indexing script updates the same records instead of inserting duplicates.

(2) Online retrieval (Retrieval): When the system receives a new ticket, it generates an embedding vector for the ticket text using the same embedding model, then performs a similarity search in Qdrant (top-2) to retrieve the most relevant policy chunks (text + metadata). The retrieved chunks are provided to the LLM as “evidence context” to constrain the final response, and the matched rule IDs and versions can be shown when escalation is triggered.

5.4 Visualisation implementation

5.4.1 Motivation and goals

Our goal is to present the end-to-end ticket handling workflow in an interactive, monitorable, and traceable way. Therefore, we built a multi-page prototype using Streamlit and used SQLite as the persistence layer to support cross-page querying and replay (SQLite.org, 2025).

5.4.2 Streamlit multi-page architecture

We built a multi-page prototype using Streamlit (Streamlit, n.d.). The main entry page (app) is responsible for creating tickets and driving the conversation workflow. Under the pages/

directory, the Dashboard, Ticket History, and Ticket Detail pages are used for overall metric monitoring, historical ticket search, and single-ticket lifecycle replay, respectively. This separation makes the responsibilities of each page clearer and reduces the development and maintenance complexity of a single large file.

(1) Interactive: The entry page uses `st.session_state.state` to maintain the conversation state (e.g., history, status, events, pending_question), and generates incremental ticket IDs using `current_ticket_id` (TICKET-1001, 1002, ...) to ensure each session can be uniquely tracked.

When the user submits input via `st.chat_input()`, the system writes the input to `base_state["query"]` and calls the backend workflow `app.invoke(base_state)`. The returned state then overwrites `st.session_state.state`, and the UI is refreshed with `st.rerun()`, forming a closed loop of “input → reasoning → display”.

To reduce redundant writes and ensure each record is a “final replay-able version”, the system writes to the database only when a ticket reaches a terminal state (CLOSED / ESCALATED):

- Overview fields are written into the `ticketstable` for Dashboard/History usage.
- The full state is serialised into `state_json` and stored in the `ticket_details` table for Detail replay and escalation explanation.

In addition, the entry page provides a “Start a new ticket” button to reset

`**st.session_state.state**` and generate a new ticket ID, making it easier to demonstrate multiple tickets in sequence.

(2) Monitorable: The Dashboard reads overview data from `st.session_state.tickets` and converts Created Time into Created Date (date) as the filtering field. The page provides a date range filter (From/To) and uses the filtered dataset `df_filtered` as the single source for both KPI calculations and chart rendering.

The KPI calculations include: Tickets (number of tickets within the selected window), Escalation rate (`mean(Escalation Needed == "Yes")`), and Auto-resolution rate (approximated as `1 - escalation_rate`).

For visualisation, we use Altair to generate two charts: a daily aggregated sentiment bar chart (padding the most recent 7 days to keep the time axis continuous), and a pie chart aggregated by Type to show the distribution of ticket categories.

(3) Traceable: The Ticket History page reuses the same date range filter (From/To) as the Dashboard. It filters `st.session_state.tickets` to produce `df_filtered`, then displays key fields in reverse chronological order (ID, Created Time, Status, Type, Sentiment, Escalation Needed),

allowing users to locate a target ticket and open its detail view.

The Ticket Detail page allows users to select a `ticket_id` from a dropdown and loads the full state from `st.session_state.ticket_details[ticket_id]`. It renders the history in chat-bubble format and displays events (or the event log) in chronological order to replay the ticket lifecycle. If the ticket status is `ESCALATED`, the page additionally reads and shows escalation evidence fields from the state, including rag-cards (RAG-matched card IDs), `ticket_summary` (ticket summary), and the escalation reason selected from `escalation_categories` by priority, which is used to explain why escalation was triggered.

5.4.3 Data & persistence

Streamlit reruns the script on every interaction, so ordinary Python variables are reset (Streamlit, n.d.). To preserve state across reruns, we store key in-session objects in `st.session_state` (e.g., `state`, `tickets`, `ticket_details`).

However, `session_state` only persists within the current session and can be lost after a refresh or restart. To support historical analytics and traceable replay, we introduced SQLite as the persistence layer. On application startup, `init_db()` creates two tables:

- `tickets` (overview table): `id`, `created_time`, `status`, `type`, `sentiment`, `escalation_needed`, used for fast filtering and aggregation in the Dashboard and History pages.
- `ticket_details` (detail table): `ticket_id`, `state_json`, where the full state (e.g., history, events, rag-cards, escalation reasons) is stored as JSON for replay in the Ticket Detail page.

For data recovery, the entry page performs two steps during initialisation:

- It loads overview records using `pd.read_sql("SELECT * FROM tickets")` and maps fields into a consistent UI format (e.g., converting 0/1 into Yes/No).
- It loads `state_json` from `ticket_details`, deserialises it, and restores it into `st.session_state.ticket_details`, so historical tickets and conversation replays remain available after a restart.

6. Results

6.1 logic results

Conversation & events

Conversation history

Hi, I'm an AI support assistant. I'll do my best to help you with your request. How may I help you today?

How long does the returns and exchanges process take?

For returns and exchanges, our processing time typically takes 7-10 business days from the date we receive the item. Once processed, the refund or replacement will be shipped to you.

Can you please confirm the order number of the item you're trying to return or exchange so I can provide more specific information related to your order?

Has your issue been resolved? Please reply Yes or No.

No

Figure 6. Example of an incorrect automatic reply without RAG support

In the test scenario without the introduction of RAG (Figure 6), we observed that there was a certain decrease in the accuracy of the answer when the system dealt with some complex or incomplete information problems. Specifically, the model lacks understanding of business details, or generates a more general or even deviating from reality response in the absence of clear contextual support. Such failures are not caused by logical errors in the overall workflow, but mainly due to the limited coverage of specific business rules and scenario constraints when the language model relies only on its own parameter knowledge. The results show that in the absence of external knowledge support, the model is more susceptible to input ambiguity and lack of context, thus increasing the risk of misjudgement.

Conversation & events

Conversation history

Hi, I'm an AI support assistant. I'll do my best to help you with your request. How may I help you today?

i want to know the payment method

Our accepted payment methods include credit and debit cards (Visa, Mastercard, Amex), bank transfers (SEPA, Wire Transfer), and PayPal. Additionally, some customers may have access to automatic payment plans or invoicing for specific subscription services.

Has your issue been resolved? Please reply Yes or No.

No

Could you describe your issue in more detail so we can better assist you?

i want to talk to a real person

We have clearly understood and recorded your question and request. We will resolve your issue as quickly as possible. Our helpful and professional staff will contact you shortly. Thank you for your patience!

Figure 7. Example of a user request for manual escalation (handover to a human agent)

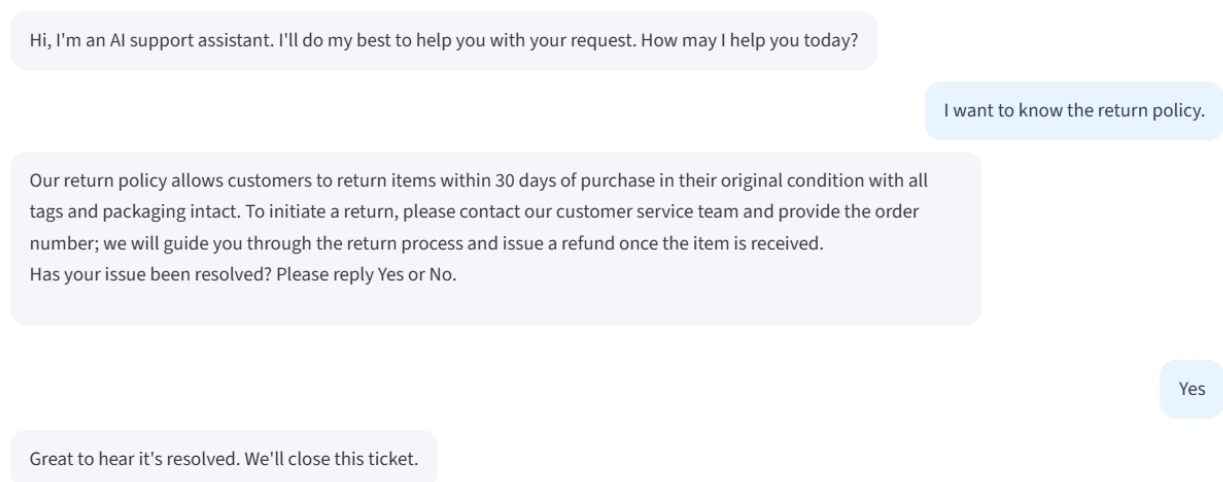


Figure 8. Example of an automatic reply and ticket closure flow

After testing different problem types and interaction styles, including explicit requests for manual escalation (Figure 7) and standard queries that can be resolved automatically with ticket closure (Figure 8), the results show that the AI-Agent’s workflow remains stable in most cases and produces appropriate outcomes based on both the user’s request and the accumulated dialogue context. In a small number of cases, the system may still generate an inaccurate response or exhibit understanding bias; this is mainly attributable to the inherent uncertainty of large language models when handling complex, ambiguous, or underspecified user inputs, rather than to the workflow design itself. To mitigate this risk, the system adopts a conservative escalation strategy and a multi-round confirmation mechanism, prioritising manual handling whenever the model output is judged risky or uncertain, thereby reducing potential negative impact and improving overall service reliability.

6.2 RAG results

This section presents the impact of RAG from two angles: (1) whether retrieval can reliably return relevant policy evidence, and (2) whether generation can provide traceable supporting information.

We also ran a small test to measure how useful RAG is for policy-based customer support. We used 10 policy-related questions (for example, return time, refund time, and billing questions) and compared two settings: No-RAG and With-RAG. We report two simple metrics: evidence rate and policy drift count (Table 2).

Compared with answering based only on the model’s internal knowledge, RAG stores policy knowledge in a vector database and retrieves relevant policy text before generation (Databricks, n.d.; Lewis et al., 2020). This allows each response to link to specific policy

cards and their versions, which can improve consistency and makes the outputs easier to audit.

6.2.1 Retrieval-side results (Qdrant)

As shown in Figure 9, we split the policy knowledge base into chunks and stored them in the Qdrant collection `support_kb_collection`. The collection shows Points (Approx) = 55, which we use as an approximate count of the indexed chunks (in our implementation, one point corresponds to one chunk).

Name	Status	Points (Approx)	Segments	Shards	Vectors Configuration (Name, Size, Distance)	Actions
support_kb_collection	● green	55	2	1	default 1536 Cosine	⋮

Figure 9. Qdrant collection for the policy knowledge base with 55 indexed chunks

As shown in Figure 10, each indexed chunk is stored as a vector point with an associated payload that contains structured traceability metadata, including the policy domain (“p”), the version date (“v”), the policy card ID (“c”), and the retrieved text snippet (“text”).

This “chunk vector + payload metadata” design follows Qdrant’s point/payload mechanism (Qdrant, n.d.).

This design allows retrieval to return not only the matched text but also the corresponding policy identifiers and version information, which can later be surfaced as evidence in the ticket detail view.

Payload:	
text	## Stage C — Additional fees and extra costs (disclosure ### BP-C1 Extra fees / add-on charges **Trigger:** - User asks about service pricing or potential additional charges. **Ask (recommended):** - Which plan/module and what feature/service they want (premium support, custom solution, upgrade, etc.). **Rules:** - Extra fee items and amounts must be documented in pricing materials or internal documentation. - Before confirming a service/subscription: clearly disclose extra fees and obtain explicit acceptance. - If customer declines: offer standard options or remove custom add-ons. **Customer output must include:** - Applicable fee items and amounts (or “requires review” if not published). - Confirmation that fees are optional/conditional and require acceptance. - Next steps if they do not accept (standard plan / remove add-ons / cancel request).
p	billing-payments-policy
v	2025-12-23
c	BP-C1

Figure 10. Example of a Qdrant point payload with traceability metadata

In our mini-evaluation (N=10), the With-RAG setting successfully returned non-empty evidence (rag-cards) for all test queries (10/10), indicating that the retrieval pipeline can

reliably surface policy snippets for downstream response grounding.

6.2.2 Generation-side results

On the generation side, the key impact of RAG is not that the workflow becomes more complex, but that responses become more controllable and explainable.

When generating an answer, the model also outputs the relevant policy card IDs (rag-cards). If the ticket is escalated (ESCALATED), the system writes “matched cards + ticket summary + escalation reason” into the ticket details as evidence fields, which supports review and handover.

From an implementation perspective, each chunk in the vector database is linked to a specific card ID *c* and version *v*. Therefore, when the system records rag-cards in a ticket, it can point back to the exact policy cards and their versions, making the citation basis traceable.

Policy drift means the answer does not match the policy text, especially when it gives wrong numbers or time periods (for example, shipping or refund days), or it gives instructions that conflict with the retrieved policy snippets. With RAG, the system can attach evidence (ragcards), so the answer is easier to check and trace.

Table 2 shows that With-RAG has less policy drift than No-RAG, which helps reduce business risk and may lower the chance of unnecessary escalation.

Metric (N=10)	No-RAG	With-RAG
Evidence rate (ragcards not None)	0/10	10/10
Policy drift count (manual check)	4/10	1/10

Table 2. Before/after metrics for Retrieval-Augmented Generation (N=10)

Figure 11 shows one typical problem in the No-RAG setting: the answer can drift away from the policy. With RAG, the system answers the same question (“How long does the returns and exchanges process take?”) using retrieved policy evidence, and it records RAG Card: RE-A0. Without RAG, no rag-cards are returned (None) and the answer gives a different processing-time range, which increases the risk of giving incorrect policy information.

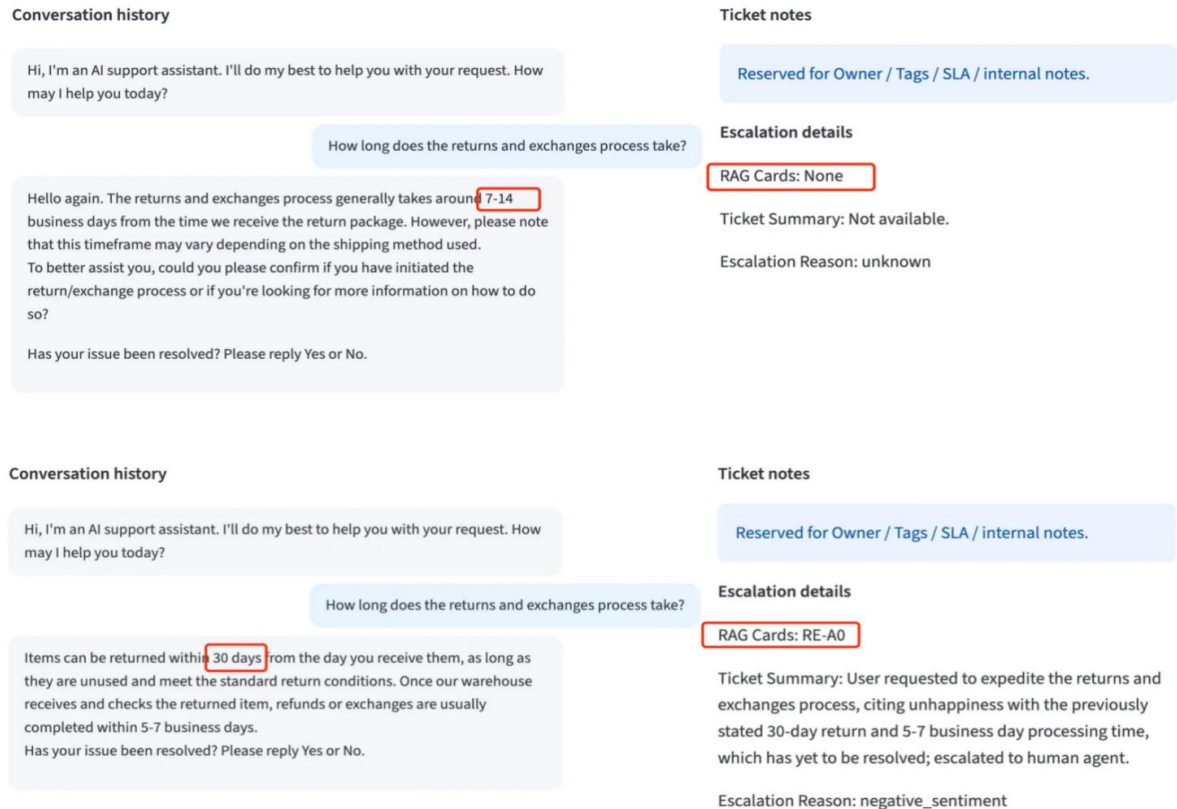


Figure 11. Comparison of ticket handling before and after enabling RAG evidence

6.3 Visualisation results

This section presents the visualisation outcomes of the Streamlit multi-page prototype: the entry page supports chat-based ticket handling, the Dashboard shows key metrics and trends, and the History/Detail pages enable historical search and single-ticket replay. Together, these pages cover the end-to-end workflow across three use cases—interaction, monitoring, and traceability—for demonstration and review.

6.3.1 Workspace

As shown in Figure 12, the entry page replicates the core workflow of a customer support ticketing product at the UI level. Users submit requests through a chat-style interface, and after each system response the user is prompted to confirm whether the issue has been resolved (Yes/No), which explicitly drives the flow to either continue clarification or reach a terminal state (closed / escalated).

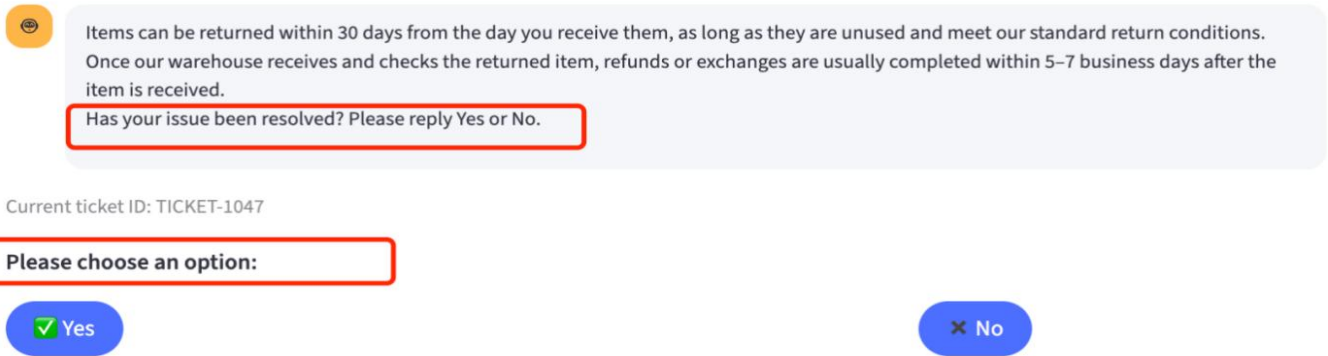


Figure 12. Workspace UI with post-response resolution driving ticket closure or escalation

As shown in Figure 13, to keep storage, analytics, and replay consistent, each ticket is assigned a unique ID (e.g., “Current ticket ID: TICKET1047”), so a conversation can be tracked as a persistent “ticket entity” rather than a one-off chat log.

Once a ticket reaches a terminal state, the page shows a clear status message (e.g., The ticket is closed) and provides a “Start a new ticket” button to quickly open the next ticket, which supports demonstrating multiple tickets with different outcomes.

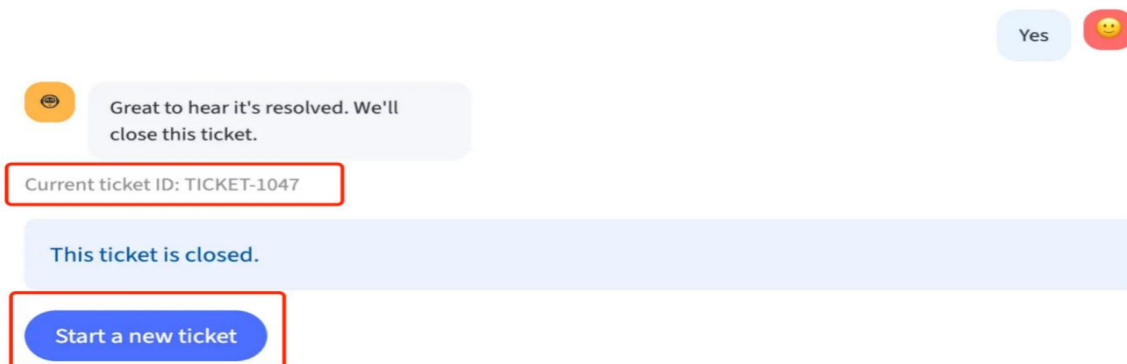


Figure 13. Ticket lifecycle UI: persistent ticket ID and “Start a new ticket” after closure

6.3.2 Dashboard

As shown in Figure 14, the Dashboard presents core operational metrics and visual analytics within a unified date window, including the total number of tickets, the auto-resolution rate, and the escalation rate. It also shows how sentiment proportions change over time and the distribution of ticket types. These metrics turn “how well the system is working” and “where performance is degrading” into observable signals.

If the escalation rate increases or the auto-resolution rate decreases, it usually indicates

weaker self-service capability and more cases requiring human intervention. This can trigger actions such as enriching the knowledge base, improving the workflow, or adjusting the model strategy.

The sentiment time series acts as an experience early-warning indicator. A sustained increase in negative sentiment may suggest a deterioration in user experience (e.g., unclear policy explanations, inconsistent time commitments, or longer processing time), and it can be followed by sampling related tickets to identify root causes.

The ticket type distribution (pie chart) helps identify the main sources of ticket volume and prioritise content improvements. If one category consistently dominates, it can be targeted first—for example by improving self-service guidance, expanding RAG policy card coverage, and refining resolution playbooks—to maximise overall impact on resolution and escalation outcomes.

Ticket Dashboard

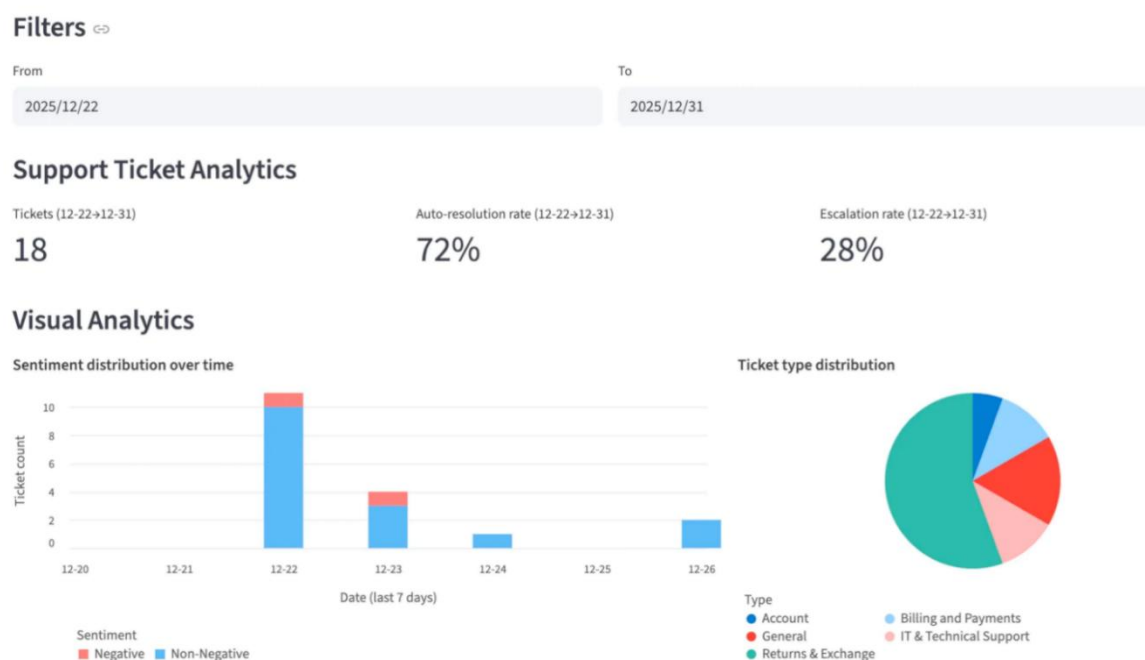


Figure 14. Ticket dashboard with operational KPIs and visual analytics

6.3.3 History + Detail

As shown in Figure 15, the Ticket History page allows users to filter tickets by a date range and displays key fields in a table view (ID, created time, status, type, sentiment, and whether

escalation was needed). This supports fast searching and locating target tickets when managing many tickets.

Ticket History

Filters

From

To

2025/12/22

2026/01/13

History Tickets

	ID	Created Time	Status	Type	Sentiment	Escalation Needed
46	TICKET-1047	01-13-2026	CLOSED	Returns & Exchange	Non-Negative	No
45	TICKET-1046	01-13-2026	ESCALATED	Returns & Exchange	Negative	Yes
44	TICKET-1045	01-13-2026	ESCALATED	Returns & Exchange	Negative	Yes
43	TICKET-1044	01-13-2026	CLOSED	Returns & Exchange	Non-Negative	No

Figure 15. Ticket history page with date-range filtering and a tabular view of key ticket fields

As shown in Figure 16, the Ticket Detail page further enables users to select a single ticket and replay the full conversation. It also summarises key ticket attributes (status, type, sentiment, and escalation requirement) on the same page, and shows escalation evidence fields (e.g., rag-cards, ticket_summary, and escalation_reason), so escalation decisions include both the outcome and the supporting context.

Ticket Detail

Select ticket ID

TICKET-1046

Ticket summary

	ID	Created Time	Status	Type	Sentiment	Escalation Needed
45	TICKET-1046	01-13-2026	ESCALATED	Returns & Exchange	Negative	Yes

Conversation & events

Conversation history

Hi, I'm an AI support assistant. I'll do my best to help you with your request. How may I help you today?

How long does the returns and exchanges process take?

Items can be returned within 30 days from the day you receive them, as long as they are unused and meet the standard return conditions. Once our warehouse receives and checks the returned item, refunds or exchanges are usually completed within 5-7 business days. Has your issue been resolved? Please reply Yes or No.

No

Ticket notes

Reserved for Owner / Tags / SLA / internal notes.

Escalation details

RAG Cards: RE-A0

Ticket Summary: User requested to expedite the returns and exchanges process, citing unhappiness with the previously stated 30-day return and 5-7 business day processing time, which has yet to be resolved; escalated to human agent.

Escalation Reason: negative_sentiment

Figure 16. Ticket detail page for replaying a single conversation with escalation evidence

7. Conclusions

This project targets a real-world customer support workflow. We designed, implemented, and tested a ticket triage and conversation handling system where AI responds first and escalates to human agents when needed, and we extended customer support from one-off Q&A to an executable and traceable workflow: each ticket goes through input validation, category identification, risk assessment, automated response, resolution confirmation, and escalation decisions, and ends in a clear final state (closed or handed over to a human agent), while the system logs decision triggers and supporting evidence for later review and iteration.

Based on our research objectives, we draw the following conclusions and explicitly answer the research questions. (RQ1: How should AI and human agents collaborate?) We define clear collaboration boundaries using a business-sensitivity and risk-driven routing strategy: high-risk tickets such as Sales/Pre-Sales, Security, and payment-related Billing/Payments are sent directly to human agents, while Account, IT Support, Returns/Exchange, General, and low-risk Billing follow an AI-first approach so human effort can focus on high-value or high-risk cases. (RQ2: When should the system escalate from AI to humans?) We propose an explainable escalation mechanism that uses four key triggers—explicit user request for a human agent, high-risk categories, negative sentiment signals, and unresolved cases after two rounds (with a resolution confirmation step)—to avoid continuing automation when user experience or compliance risk may worsen; to make escalation signals reliable, we built a 500-sample human-labelled Gold Set and compared a baseline model with two LLMs, finally choosing the model with higher negative-class recall (llama-3.1-8b-instant: Recall Neg 0.9423, F1 0.9108) to reduce the risk of missed escalations. (RQ3: How should response strategies vary by issue type, and what is the impact of RAG?) For AI-first tickets, we introduce Retrieval-Augmented Generation (RAG) to retrieve relevant policy evidence before generating responses, and we record policy card IDs (rag-cards) and rationale fields during escalation/handover to improve traceability and handoff efficiency; in a comparative test of 10 policy-related questions, adding RAG achieved an evidence rate of 10/10 and reduced policy drift from 4/10 to 1/10, suggesting that evidence-grounded generation can reduce policy deviations and improve consistency and auditability.

From an operations perspective, this system is closer to a “governable customer support automation pipeline” than a simple auto-reply bot. It keeps evidence fields (rag-cards, ticket summary, escalation reasons) and provides History/Detail playback to support quality checks, audit accountability, and smooth human takeover; it also visualises key metrics such as auto-resolution rate and escalation rate on a dashboard, enabling ongoing monitoring and

policy/strategy updates. To enable these capabilities, we use a state-machine/workflow-based agent design (LangGraph/StateGraph), split classification, generation, confirmation, and escalation into independently testable nodes, and maintain a shared ticket state across multi-turn conversations so the system behaviour is controlled, traceable, and reproducible.

Although the system forms an end-to-end closed loop, it is still a prototype. Data mismatch and missing context may affect generalisation, and LLM outputs can be unstable in real deployments. Therefore, our design prioritises controllability and risk-aware escalation over maximum automation, which may increase human workload. Future work will improve robustness and industrial readiness, as detailed in Section 8.

8. Future work

Although this project has achieved a complete closed loop from user dialogue, work order generation, automatic reply to upgrade labour, and has the basic statistical board and work order tracking capabilities, the system is still a prototype at present. To become a large-scale and sustainable industrial-level customer service product, there are still several key gaps, which are mainly reflected in the four aspects of data, model reliability, compliance security and commercialisation mechanism.

1. Insufficient coverage of data and real business

At present, the design of the system is mainly verified in limited data sets and controllable test scenarios. The types of questions in the real customer service environment are more complex, including a large number of long questions, mixed intentions (such as asking the payment method first and then complaining about deduction), non-standard expressions, colloquial input, and even malicious input. The lack of an environment from real business will lead to unstable generalisation ability of the system in new scenarios.

In the future, work can introduce work order data and online feedback mechanisms that are closer to the real business. For example, manual customer service fills in the labels such as "final solution/real category/whether to upgrade" after processing the upgrade work order, which is used for continuous iteration.

2. There is still uncertainty in the model output, and there is a lack of industrial-level reliability constraints.

The system currently relies on LLM to complete classification, emotional judgement and

response generation. LLM naturally has output fluctuations, and there will be risks such as occasional deviation from requirements and overconfidence in fuzzy problems. Course items can accept "occasional instability", but industrial products need a stable service experience, otherwise it will directly affect user satisfaction and brand credibility.

Future work should strengthen deterministic control, such as: introducing stricter structured output constraints, confidence assessment and compliance regulatory strategy.

3. Lack of industrial-level security and compliance capabilities

Industrial customer service products must face privacy, compliance and security issues, such as: users may enter personal information, payment information, order numbers; the system needs to meet the requirements of data storage and access control; and it also needs to deal with the risks of prompt injection, over-the-right instructions, data leakage and so on. Prototypes usually only focus on running functions, but commercial products must take security as the default premise.

Future work should be completed: PII (personal sensitive information) identification and desensitisation, access rights and auditing, log compliance preservation, and security filtering of input and output.

4. Lack of operational and commercialisation-oriented index system and cost control

Truly profitable customer service products must not only "be able to answer", but also prove that "can reduce costs and increase efficiency". This means that clear operational indicators are needed: automatic resolution rate, false upgrade rate, false shutdown rate, average processing time, user satisfaction, labour saving hours, etc., and the need to control the reasoning cost and LLM invocation delay. Although the current system has a basic statistical board, a complete index evaluation system and cost optimisation strategy have not been established.

Future work can be further: integrate the results of each dialogue and work order into the quantifiable index system; build an A/B test framework to compare different strategies; introduce multi-model routing (lightweight models for simple problems and high-parameter models for complex problems). Optimise the LLM cache, concurrency and call frequency of the system to reduce the processing cost of tickets and meet the requirements of commercialisation.

Acknowledgements

On the occasion of the successful completion of this summer project, we would like to express our sincere thanks to all the teachers and students who have given us their guidance and support.

First of all, we would like to express my sincere thanks to the two project supervisors - Dr. Phil Davies and Dr. Yiwen Lyu. The two tutors gave continuous and meticulous guidance in terms of project topic selection, research direction control, technical route suggestions and result presentation. When we encountered key bottlenecks and uncertainties, they also provided timely, professional and inspiring feedback to help the team continuously calibrate goals and improve the feasibility and academic Normative. In the process of project promotion, they have provided valuable support in terms of demand discussion, technical exchange, tool use experience and resource sharing, so that we can complete system construction and iterative optimisation more efficiently.

At the same time, we would like to thank James Williams, Cameron Bell and other members of the Master of Applied Data Science team at the University of Canterbury.

In addition, we also make the following statement about the technical stack and auxiliary tools used in this project: the main programming language of this project is Python, the development environment uses Visual Studio Code (VS Code), and the cloud platform uses Microsoft Azure. Re, the large language model (LLM) call service uses the Groq platform. At the same time, OpenAI's ChatGPT is used as a project auxiliary tool, mainly for reporting language and syntax optimisation, bug troubleshooting and debugging support, and project progress planning and sorting to improve the efficiency and quality of research and development and document work.

Thanks again to all the teachers and students who have supported and helped us. Your guidance and cooperation are an important guarantee for the implementation of this project on schedule.

References

- Davenport, T. H., Guha, A., Grewal, D., & Bressgott, T. (2020). *How artificial intelligence will change the future of marketing*. *Journal of the Academy of Marketing Science*, 48(1), 24–42. <https://link.springer.com/article/10.1007/s11747-019-00696-0>
- Mohammad, S. M. (2016). *Sentiment analysis: Detecting valence, emotions, and other affectual states from text*. In H. L. Meiselman (Ed.), *Emotion measurement* (pp. 201–237). Elsevier. <https://doi.org/10.1016/B978-0-08-100508-8.00009-6>
- Hutto, C. J., & Gilbert, E. (2014). *VADER: A parsimonious rule-based model for sentiment analysis of social media text*. *Proceedings of the International AAAI Conference on Web and Social Media*, 8(1), 216–225. <https://ojs.aaai.org/index.php/ICWSM/article/view/14550>
- Databricks. (n.d.). *Retrieval augmented generation (RAG)*. Retrieved January 15, 2026, from <https://www.databricks.com/glossary/retrieval-augmented-generation-rag>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv. <https://doi.org/10.48550/arXiv.2005.11401>
- McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11), 205. <https://doi.org/10.21105/joss.00205>
- Qdrant. (n.d.). *Points*. *Qdrant Documentation*. Retrieved January 15, 2026, from <https://qdrant.tech/documentation/concepts/points/>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982–3992). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>
- Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021* (pp. 3784–3803). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.findings-emnlp.320>

Sentence-Transformers. (n.d.). *all-MiniLM-L6-v2* [Model card]. Hugging Face. Retrieved January 15, 2026, from <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

Streamlit. (n.d.). *Add statefulness to apps (Session State)*. Streamlit Docs. Retrieved January 15, 2026, from <https://docs.streamlit.io/develop/concepts/architecture/session-state>

SQLite. (2025, November 13). *About SQLite*. <https://www.sqlite.org/about.html>

Raux, A., & Eskenazi, M. (2009). A finite-state turn-taking model for spoken dialog systems. *Proceedings of the 2009 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 629–637. <https://aclanthology.org/N09-1071/>

LangChain. (n.d.). *Graph API overview*. Docs by LangChain. Retrieved January 15, 2026, from <https://docs.langchain.com/oss/python/langgraph/graph-api>

Gao, S., Sethi, A., Agarwal, S., Chung, T., & Hakkani-Tür, D. (2019). *Dialog state tracking: A neural reading comprehension approach*. arXiv. <https://doi.org/10.48550/arXiv.1908.01946>

Kiepuszewski, B., ter Hofstede, A. H. M., & van der Aalst, W. M. P. (2003). *Fundamentals of control flow in workflows*. *Acta Informatica*, 39(3), 143–209. <https://doi.org/10.1007/s00236-002-0105-4>

Appendices

Appendix 1: Key Technical Terms (Glossary)

Term	Short definition (English)
AI-Agent	A system that can perceive context, make decisions, and take actions (often via tools) to achieve goals.
LLM (Large Language Model)	A neural language model trained on massive text data to understand and generate natural language.
RAG (Retrieval-Augmented Generation)	A method that retrieves relevant external knowledge first, then uses it to ground LLM generation.
Retriever	A component that selects the most relevant documents/snippets for a given query.
Embedding	A vector representation of text that captures semantic meaning for similarity comparison.
Vector Database (Vector DB)	A database optimised for storing and searching embeddings efficiently.
Semantic Search	Search based on meaning similarity (embeddings) rather than exact keyword matches.
Hybrid Search	A combination of semantic search (vector) and keyword/structured filtering approaches.
Metadata Filtering	Restricting results using structured fields (e.g., tag, category, date, source).
Generation	The LLM step that produces natural-language output from instructions and context.
Prompt	The instruction/input text that guides an LLM's behaviour and output.
Few-shot Learning	Steering model behaviour by providing a small number of examples in the prompt.
LangChain	A framework for building LLM applications with chains, tools, memory, and integrations.
NLP (Natural Language Processing)	The field focused on computational methods for understanding and generating human language.
ANN (Approximate Nearest Neighbour)	Fast approximate methods for nearest-vector search at scale.
Low Latency	A property of systems that respond with minimal delay.

Cloud Hosting	Running services on cloud infrastructure rather than local machines.
Self-hosting	Deploying and operating a system on your own infrastructure.
Python SDK	A Python client library used to interact with a platform or service API.
REST API	An HTTP-based interface for programmatic access to services.
gRPC	A high-performance RPC framework using Protocol Buffers for typed messages.
Git	A distributed version control system for tracking changes in source code.
GitHub	A platform for hosting Git repositories and collaborating on code.
Repository (Repo)	A version-controlled project space containing code, history, and documentation.
Codespace	A cloud-hosted development environment (commonly associated with GitHub).
Chroma	A lightweight vector database often used for prototyping.
Pinecone / Milvus / Weaviate	Popular vector database solutions with different scaling and deployment trade-offs.
Qdrant	A vector database supporting similarity search and metadata filtering.

Appendix 2: GitHub repository

<https://github.com/UC-25-Summer-DATA601-AI-Agent/UC-25-Summer-DATA601-AI-Agent>